

VIO / LVIO / MSCKF — 평생 쓰는 사고방식 & 암기 cheat sheet

작성일 2026-05-20 대상 본인 졸업 연구 (Stereo + IMU gravity + LiDAR plane → LC-EKF → TC MSCKF) 의 평생 reference **사용법** 새 알고리즘·논문·코드 만날 때, “어느 카탈로그에 속하지?” 로 분해하는 도구로 사용 **PDF 추출** 본 문서 마지막 부록 참고

0. 본 문서의 목적과 구조

VIO/LVIO/MSCKF 류의 작업을 백지에서 할 수 있는 사람의 머릿속에는 **두 가지 자산**이 있습니다.

1. **사고방식** — 새 문제를 익숙한 부품의 조립으로 분해 하는 흐름
2. **카탈로그** — 외워둔 수식·라이브러리·알고리즘 패턴 (외울 가치 있는 것만)

본 문서는 두 자산을 본인 연구 컨텍스트 (Stereo+IMU+LiDAR-plane VIO → TC MSCKF 전환) 로 묶어 정리합니다.

- Part 1 사고방식 – 백지 → 함수 5 단계
 - Part 2 암기: 수식 – 회전 / IMU / EKF / MSCKF
 - Part 3 암기: 라이브러리 / 코드 패턴
 - Part 4 암기: 알고리즘 패턴
 - Part 5 본인 연구 적용 – LC-EKF → TC MSCKF 전환 사고
- 부록 PDF 추출 / 추가 학습 자료

각 섹션은 사용자 선호 형식 (요약 → 수식/코드 → 직관 → EKF 의미 → C++ 노트) 의 다층 블록입니다.

Part 1 — 사고방식 (생각의 흐름)

1.1 백지에서 함수 한 개 짜는 5 단계

요약 키보드 두드리기 전에 머릿속 모델이 80% 완성. 키보드는 transcription.

1. signature 먼저 결정 – 입력/출력 타입, 반환값 형태
2. 박스 다이어그램으로 분해 – 5 박스 이내가 이상적
3. 박스마다 표준 부품 매핑 – 카탈로그에서 꺼냄
4. 박스마다 엣지 케이스 – 입력 비정상 / 수치 발산 / 빈 컨테이너
5. 짜기 시작 – 30 초 ~ 2 분 안에 초안

직관 외국어 작문과 동일. 단어(언어) → 어휘(stdlib) → 문장(알고리즘) → 문단 (도메인) → 글쓰기 직감(빌드/디버그) 5 층 위에서 작업.

EKF 의미 EKF predict 함수 짜기? signature: state, input, dt → new state 박스 4 개로 분해 (회전 적분, 속도 적분, 위치 적분, bias 유지). 이 박스는 어떤 VIO 프로젝트에도 똑같이 등장.

1.2 모든 VIO 알고리즘이 따르는 3 분법

요약 어떤 VIO 논문·코드도 본인 머릿속을 3 개 슬롯 으로 정리해두면 30% 빨리 읽힘.

슬롯	질문	본 연구 예시
State	무엇을 추정하는가?	$[\mathbf{p}, \mathbf{v}, \mathbf{q}, \mathbf{b}_a, \mathbf{b}_g, \mathbf{p}_{clone_i}]$
Input	무엇이 들어와 state 를 밀어붙이나?	IMU acc, gyro (predict 단계)
Measurement	무엇이 들어와 state 를 교정하나?	Stereo feature, LiDAR plane (update 단계)

직관 Input 과 Measurement 의 차이 = "예측" 과 "교정" 의 차이. IMU 는 예측에만, 카메라/LiDAR 는 교정에만 (보통). 헛갈리면 **노이즈를 곱한 Jacobian 이 어디로 전파되나** 를 따라가면 명확해짐.

EKF 의미 Input $\rightarrow$ Process noise Q , Measurement $\rightarrow$ Measurement noise R . 두 노이즈 슬롯이 헛갈리면 모든 게 무너짐.

1.3 좌표계·시간축·단위 sanity 체크 (Pre-flight)

요약 VIO 디버깅의 80% 는 프레임/타임/단위 셋 중 하나가 어긋난 거. 코드 짜기 전 반드시 명시.

항목	본 연구 컨벤션
World frame	KITTI 첫 IMU pose 기준 (gravity-aligned)
Body frame	IMU 중심 (FLU 또는 KITTI 본)
Camera frame	OpenCV 본 (z-forward, x-right, y-down)
LiDAR frame	KITTI Velodyne 본
시간 단위	second (모든 dt)
길이 단위	meter
각도 단위	radian (내부), 표시할 때만 deg
Quaternion 컨벤션	Hamilton, $[w, x, y, z]$ 또는 Eigen 의 $[x, y, z, w]$ $\leftarrow$ 혼동 주의

C++ 노트 Eigen Quaterniond::Quaterniond(w, x, y, z) 생성자 vs 내부 저장 순서가 다름. q.coeffs() 는 $[x, y, z, w]$, 생성자는 $[w, x, y, z]$. **이거 한 번 틀리면 3 일 없음.**

직관 디버깅 1 단계 = "프레임 어디서 어디로?" 외쳐보기. T_{WB} , T_{BC} , T_{CL} 같은 첨자는 주어 $\rightarrow$ 표적 으로 읽음 (W from B, B from C, C from L). 곱하면 사라지는 글자가 옳음: $T_{WB} \cdot T_{BC} = T_{WC}$.

1.4 알고리즘 / 논문 읽을 때 head 5 분 절차

1. State 무엇? Input 무엇? Measurement 무엇? (Part 1.2 3 분법)
2. Predict 모델은? (보통 IMU)
3. Update 모델은? (vision? plane? GPS?)
4. State 어떻게 marginalize / clone 하나?
5. 어디서 outlier 거르나?

이 5 개 칸 채우면 어떤 VIO 논문도 통째로 정리됨. ov_plane / MSCKF / VINS-Mono / Kimera / OpenVINS 다 동일.

Part 2 — 필수 암기 (수식)

암기 기준: 외워둬야 읽는 속도 가 빨라지는 것만. 유도가 필요한 건 책 옆에 둘 것.

2.1 회전 표현 3 가지 — 평생 가지고 다님

요약 Rotation matrix R , quaternion q , axis-angle $\theta = \phi \mathbf{u}$ 세 표현은 머릿속에서 자유롭게 왕복 가능해야 함.

$R \in SO(3)$	9 numbers, 정직, 곱셈 비쌈
$q \in S^3$	4 numbers, 효율, 보간 자연스러움
$\theta \in \mathbb{R}^3$ ($so(3)$)	3 numbers, error/perturbation 의 자연 표현

핵심 변환 4 개 (외움)

출발	도착	수식
θ	R	$R = \exp([\theta]_{\times}) = I + \frac{\sin \phi}{\phi} [\theta]_{\times} + \frac{1 - \cos \phi}{\phi^2} [\theta]_{\times}^2$ (Rodrigues)
R	θ	$\phi = \arccos \frac{\text{tr}(R) - 1}{2}, \mathbf{u} = \frac{1}{2 \sin \phi} (R - R^T)^{\vee}$
q	R	$R = (q_w^2 - \mathbf{q}_v^T \mathbf{q}_v) I + 2 \mathbf{q}_v \mathbf{q}_v^T + 2 q_w [\mathbf{q}_v]_{\times}$
작은 회전	q	$q \approx [1, \frac{1}{2} \theta]$ (perturbation linearization 의 출처)

직관 Error-state EKF 에서 회전 오차는 항상 $\delta \theta \in \mathbb{R}^3$ 으로 저장. 이유: 3 DoF 인데 $R(9)$ 이나 $q(4)$ 를 state 로 쓰면 차원 잉여 $\rightarrow$ 공분산 행렬 특이.

C++ 노트 Sophus::S03d 가 위 변환을 안전하게 처리. 직접 팔 때 $\phi \rightarrow 0$ 인 경우 (Taylor expand) 챙기기. 안 그러면 NaN.

Skew-symmetric 연산 (외움)

$$[\mathbf{v}]_{\times} = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix}, \quad [\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}$$

암기 트릭: 우상 대각에서 시계 반대 방향으로 $-z, +y, -x$.

2.2 좌표 변환 chain — 첨자 cancel 규칙

요약 $T_{AB} \cdot T_{BC} = T_{AC}$. 가운데 첨자 (B) 가 사라지면 옳음.

$$\mathbf{p}_W = T_{WB} \cdot \mathbf{p}_B = T_{WB} \cdot T_{BC} \cdot \mathbf{p}_C$$

역변환: $T_{AB}^{-1} = T_{BA}$. 점이 어느 frame 에서 표현돼 있는지 항상 추적.

C++ 노트 변수명에 frame 첨자 박아라: T_W_B, p_C_feat, R_BC. 첨자 없는 변수명은 디버깅 지옥의 시작.

2.3 IMU 모델 — 평생 같은 모양

요약 가속도계 $\mathbf{a}_m$, 자이로 ω_m 측정값은 **참값 + bias + noise + 가속도엔 중력 빼기**.

$$\omega_m = \omega + \mathbf{b}_g + \mathbf{n}_g, \quad \mathbf{a}_m = R_{BW}(\mathbf{a} - \mathbf{g}_W) + \mathbf{b}_a + \mathbf{n}_a$$

Continuous-time predict (외움 — predict 의 뼈)

$$\begin{aligned} \dot{\mathbf{p}}_W &= \mathbf{v}_W, & \dot{\mathbf{v}}_W &= R_{WB}(\mathbf{a}_m - \mathbf{b}_a) + \mathbf{g}_W, & \dot{q}_{WB} &= \frac{1}{2} q_{WB} \otimes [0, \omega_m - \mathbf{b}_g] \\ \dot{\mathbf{b}}_a &= \mathbf{n}_{b_a}, & \dot{\mathbf{b}}_g &= \mathbf{n}_{b_g} \quad (\text{random walk}) \end{aligned}$$

직관 회전식의 $\frac{1}{2}$ 는 quaternion 의 double cover 때문. 외적 $\omega \times$ 를 quaternion 곱 $q \otimes [0, \omega]$ 로 옮기면서 절반이 들어감.

EKF 의미 Predict 의 처음 5 줄. 본 연구 imu_propagator.cpp 가 정확히 이 식을 이산화. RK4 또는 mid-point 적분이 보통.

2.4 EKF 6 식 — 일생 외우는 6 줄

요약 Predict 2 줄 + Update 4 줄. 모든 KF 변종 (EKF, UKF, ESKF, MSCKF, iEKF) 이 이 6 줄의 변형.

Predict

$$\begin{aligned} \hat{\mathbf{x}}^- &= f(\hat{\mathbf{x}}, \mathbf{u}) \\ P^- &= F P F^T + G Q G^T \end{aligned}$$

Update

$$\mathbf{y} = \mathbf{z} - h(\hat{\mathbf{x}}^-) \quad (\text{innovation})$$

$$S = HP^-H^T + R \quad (\text{innovation cov})$$

$$K = P^-H^TS^{-1} \quad (\text{Kalman gain})$$

$$\hat{\mathbf{x}}^+ = \hat{\mathbf{x}}^- + K\mathbf{y}, \quad P^+ = (I - KH)P^- \quad (\text{state, cov update})$$

직관 K = "measurement 를 얼마나 믿을지" 의 가중치. R 작으면 ($\rightarrow$ 좋은 센서) K 큼, R 크면 작음. Bayes 의 직관 그대로.

C++ 노트 공분산 update 는 **Joseph form** $P^+ = (I - KH)P^-(I - KH)^T + K RK^T$ 가 수치적으로 안정. 단순 $(I - KH)P^-$ 는 P 가 음의 정칙성 잃을 수 있음. 본 연구도 결국 Joseph form 으로 옮길 가능성 큼.

Error-state (ESKF) 보너스

Quaternion 같이 manifold 위에 있는 state 는 error-state 로 KF 적용:

$$\mathbf{x} = \hat{\mathbf{x}} + \delta\mathbf{x}, \quad \delta\mathbf{x} \in \mathbb{R}^n$$

KF 는 $\delta\mathbf{x}$ 위에서 돌고, $\delta\mathbf{x}$ reset 한 다음 nominal $\hat{\mathbf{x}}$ 에 흡수. 본 연구 LC-EKF 가 이 구조.

2.5 Stereo Reprojection — Vision update 의 뼈

요약 Feature 의 3D 위치 $\rightarrow$ 좌/우 카메라 픽셀. 잔차는 픽셀 단위.

$$\mathbf{p}_C = T_{CW}\mathbf{p}_W, \quad \pi(\mathbf{p}_C) = \begin{bmatrix} f_x X/Z + c_x \\ f_y Y/Z + c_y \end{bmatrix}$$

좌/우 disparity:

$$\text{disp} = f_x \cdot b / Z_C$$

b = baseline, Z_C = 카메라 frame 에서 깊이.

EKF 의미 Measurement model h 가 위. Jacobian $H = \partial\pi/\partial\mathbf{p}_C \cdot \partial\mathbf{p}_C/\partial\mathbf{x}$ 두 단계 chain rule. MSCKF 의 핵심 계산.

2.6 Plane / Point-to-Plane — LiDAR 측정 모델

요약 평면 $\mathbf{n}^T \mathbf{p} + d = 0$ ($\|\mathbf{n}\| = 1$). 점과 평면 거리 = $\mathbf{n}^T \mathbf{p} + d$ (부호 있음, 절대값이 진짜 거리).

본 연구의 LiDAR plane constraint:

$$\mathbf{r}_{\text{plane}} = \mathbf{n}_W^T \mathbf{p}_W + d$$

$\mathbf{p}_W$ 가 평면 위에 있어야 한다는 게 측정. $\mathbf{p}_W$ 는 state (IMU pose) 와 extrinsic 으로 표현되므로 자연스럽게 EKF update.

직관 Floor 평면이라면 $\mathbf{n}_W \approx [0, 0, 1]$, $d = -h_{\text{IMU}}$. Roll/pitch + 높이 3 자유도를 한 번에 묶어줌. 평지 환경에서 LC-EKF drift 의 핵심 억제력.

EKF 의미 Plane 자체를 state 에 넣을 수도 (4 DoF, quaternion-like over-parameterization 주의), 외부에서 고정 평면으로 줄 수도. 본 연구는 segmentation 으로 매 frame 추정 한 평면을 measurement 로 던지는 구조.

2.7 MSCKF — Null-space Projection

요약 Feature 위치를 state 에 안 넣고도 multi-view constraint 를 update 로 쓰는 트릭. State dimension 절약 + 일관성(consistency) 좋음.

한 줄 직관

Feature $\mathbf{p}_f$ 가 M 개 카메라 pose 에서 관측되면, 잔차는 $\mathbf{r} \in \mathbb{R}^{2M}$. Jacobian 은 H_x (pose 들) 과 H_f (feature 위치) 로 갈라짐:

$$\mathbf{r} \approx H_x \delta \mathbf{x} + H_f \delta \mathbf{p}_f + \mathbf{n}$$

Trick: H_f 의 left null-space V 를 양변에 곱하면 $\mathbf{p}_f$ 의존성 소거:

$$V^T \mathbf{r} = V^T H_x \delta \mathbf{x} + V^T \mathbf{n}$$

이제 표준 EKF update.

EKF 의미 State 에 feature 안 넣어도 됨 $\rightarrow O(n^2)$ 가 아닌 $O(n)$ 에 가까운 비용. 이게 MSCKF 가 EKF-SLAM 보다 빠른 이유.

C++ 노트 Null-space 는 QR 분해의 Q_2 또는 SVD 의 마지막 left singular vectors. Eigen HouseholderQR 또는 JacobiSVD. ov_plane UpdaterMSCKF.cpp 가 정확히 이 단계를 가지고 있음. 다음 분석 타깃.

Part 3 — 필수 암기: 라이브러리 / 코드 패턴

3.1 Eigen — 자주 쓰는 25 개

#	호출	의미
1	<code>Eigen::Vector3d v(1,2,3);</code>	3D 벡터
2	<code>Eigen::Matrix3d M;</code>	3×3 행렬
3	<code>Eigen::MatrixXd::Zero(r,c)</code>	0 행렬
4	<code>Eigen::MatrixXd::Identity(n,n)</code>	단위 행렬
5	<code>M.transpose()</code>	전치
6	<code>M.inverse()</code>	역행렬 (작은 행렬만, 큰 건 solve 로)
7	<code>M.ldlt().solve(b)</code>	대칭 PD 시스템
8	<code>M.householderQr().solve(b)</code>	일반 시스템
9	<code>M.block<r,c>(i,j)</code>	컴파일 타임 블록
10	<code>M.block(i,j,r,c)</code>	런타임 블록
11	<code>v.norm(), .squaredNorm()</code>	L2
12	<code>v.normalized()</code>	단위 벡터
13	<code>a.dot(b), a.cross(b)</code>	내적·외적
14	<code>Eigen::Quaterniond q(w,x,y,z)</code>	생성자 순서 !
15	<code>q.toRotationMatrix()</code>	$q \rightarrow R$
16	<code>Eigen::AngleAxisd(angle, axis)</code>	axis-angle
17	<code>M.selfadjointView<Lower>()</code>	대칭 가정
18	<code>M.eigenvalues()</code>	고유값 (PSD check 용)
19	<code>M.determinant()</code>	det
20	<code>M.trace()</code>	tr
21	<code>Eigen::Map<Vector3d>(ptr)</code>	외부 메모리 wrap
22	<code>M.noalias() = A * B</code>	aliasing 회피 (속도)
23	<code>M.array()</code>	element-wise 모드
24	<code>(v.array() > 0).count()</code>	조건 카운트
25	<code>'JacobiSVD svd(M, ComputeFullUW</code>	<code>ComputeFullV)'</code>

C++ 노트 Eigen 객체를 STL 컨테이너에 넣을 땐 `Eigen::aligned_allocator` 필요 (`std::vector<Vector3d, Eigen::aligned_allocator<Vector3d>>`). 안 하면 SIMD alignment 깨져서 segfault.

3.2 C++ 관용구 — VIO 코드에서 매번 보는 것

#	관용구	왜
1	<code>const T&</code> 파라미터	복사 회피 + 불변 보장
2	<code>T&&</code> move semantics	큰 행렬 반환 시
3	<code>RAll (ofstream, lock_guard)</code>	자원 자동 해제
4	<code>.hpp</code> 선언 / <code>.cpp</code> 정의 분리	컴파일 시간 + ODR
5	<code>std::vector + reserve()</code>	알 수 있으면 미리 할당
6	<code>std::optional<T></code>	"값 또는 없음"
7	<code>std::variant<A,B,C></code>	tagged union (segment label 등)
8	<code>enum class : uint8_t</code>	타입-안전 enum + 메모리 절약
9	<code>static_cast<T>, reinterpret_cast<char*></code>	변환 의도 명시
10	<code>assert(...)</code>	invariant 명시 (debug 만)
11	<code>[[nodiscard]]</code>	반환값 무시 방지
12	<code>constexpr</code> 상수	컴파일 시 계산
13	<code>template + typename T</code>	generic 컨테이너
14	<code>range-based for for (const auto& x : v)</code>	순회 표준
15	<code>std::move(local)</code>	마지막 사용 시

3.3 디버깅 도구 7 종

1. `assert(P.isApprox(P.transpose()));` // 공분산 대칭 체크
2. `assert(P.ldlt().info() == Eigen::Success);` // PSD 체크
3. `assert(std::isfinite(x.norm()));` // NaN/Inf 박멸
4. `std::cout << "DBG[" << __LINE__ << "]" << v.transpose() << "\n";`
5. CSV 로그 (Python pandas + matplotlib 분석)
6. ROS rviz / Rerun.io 로 trajectory + landmark 동시 시각화
7. ground truth 와 ATE/RPE 비교 (evo 패키지)

직관 디버깅은 측정 이지 추측 이 아님. NaN 발생 시 가장 빠른 길: 매 step 끝에 sanity check 박고 어디서 NaN 처음 나는지 binary search.

Part 4 — 필수 암기: 알고리즘 패턴

4.1 Predict → Update 루프 (모든 VIO 의 척추)

while (data 남았으면):

1. 다음 sensor packet 가져오기 (IMU 또는 vision/LiDAR)
2. IMU 면 → propagate state to packet.time (Predict)
3. vision/LiDAR 면 → 측정 모델 + Jacobian → Update
4. (MSCKF 면) state cloning / marginalization
5. (선택) sanity check, NaN check, cov PSD check

본 연구의 모든 main loop 가 이 패턴.

4.2 State Cloning (MSCKF)

요약 새 카메라 frame 들어올 때마다 현재 IMU pose 를 **state vector** 에 복사 → "sliding window" 형성. Feature 가 충분히 관측되면 그 window 의 모든 pose 를 한 번에 update.

```
clone:      x ← [x; T_WC_now]    (state augmented)
propagate:  IMU 들어오면 가장 최근 IMU pose 만 predict
update:     feature track 끝나면 window 전체로 MSCKF update
marginalize: window 가 차면 가장 오래된 clone 제거 (Schur complement)
```

EKF 의미 공분산 행렬도 같이 augmented: 새 행/열은 IMU pose 와 상관 을 가짐. 단순 zero-pad 면 정보 손실.

C++ 노트 ov_plane 에서 Propagator::propagate_and_clone() 같은 함수가 정확히 이 단계. **다음 분석 핵심.**

4.3 Sliding Window vs Filter

패러다임	대표	비용	정확도
Filter (EKF/MSCKF)	OpenVINS, MSCKF	$O(n) \sim O(n^2)$	보통
Sliding Window BA	VINS-Mono, ORB-SLAM3	$O(n^3)$ inner, $O(\text{window}^2)$ outer	더 좋음
Full-batch BA	offline COLMAP	$O(N^3)$	최고

직관 실시간 = filter, 오프라인 정확도 = BA. 하이브리드도 많음 (MSCKF + occasional BA).

4.4 RANSAC (Floor/Wall 추출용)

```
best_inliers = 0
for i in 1..N_iter:
    1. 무작위 minimal set 뽑기 (평면이면 3 점)
    2. 모델 fitting (외적으로 평면 4 계수)
    3. 모든 점에 대해 distance < threshold 인 inlier 수
    4. best 보다 많으면 갱신
return best 모델 + inliers
```

직관 Outlier 비율 ϵ , minimal set 크기 s , 성공 확률 p 라 할 때 필요한 반복 수: $N = \log(1 - p) / \log(1 - (1 - \epsilon)^s)$. 외위들 가치 있는 식.

4.5 Outlier Rejection — Chi-square test

요약 Innovation $\mathbf{y}$ 와 그 공분산 S 가 있을 때 Mahalanobis 거리

$$\chi^2 = \mathbf{y}^T S^{-1} \mathbf{y}$$

가 자유도 만큼의 chi-square 분포 임계값을 넘으면 outlier 로 버림.

EKF 의미 Update 전 "이 measurement 가 정상인가?" 의 표준 게이트. 본 연구 stereo update / LiDAR plane update 둘 다에 필수.

C++ 노트 임계값은 자유도별로 표로 외움. dof=2 (stereo pixel): 5.99 (95%), dof=3 (3D point): 7.81, dof=1 (scalar): 3.84.

Part 5 — 본인 연구 적용 (LC-EKF → TC MSCKF 전환)

5.1 현 상태 (2026-05-20)

완료: Stereo + IMU + LiDAR floor/wall segmentation

LC-EKF 1 차 구현, KITTI seq00 ATE ≈ 153 cm

다음: Seg-aided TC MSCKF 로 전환

ov_plane (RPNG, ICRA 2023) UpdaterMSCKF.cpp 분석 시작

5.2 LC vs TC — 사고방식 차이

항목	LC-EKF (현재)	TC MSCKF (다음)
Vision module	외부 (VO) → pose 만 EKF 로 전달	feature track 자체를 EKF measurement
LiDAR module	평면을 외부 추정 → plane constraint 전달	동일하게 plane 을 measurement
State 크기	IMU 15 + bias 6 = 21	+ N 개 camera clone $\times 6 \rightarrow \sim 50+$
정보 손실	Vision 의 공분산이 inflate (compressed pose)	픽셀 잔차 직접 → 정보 보존
코드 난이도	중	상 (state cloning, null-space projection)

직관 LC = 정수기 거친 물, TC = 직접 마시는 물. 정보 보존 $\uparrow$, 복잡도 $\uparrow$.

5.3 전환 시 추가 암기 카탈로그

- ☐ State cloning 메커니즘 (Part 4.2)
- ☐ Null-space projection (Part 2.7)
- ☐ Stereo feature triangulation (DLT / Gauss-Newton refine)
- ☐ Outlier rejection (Part 4.5)
- ☐ FEJ (First-Estimates Jacobian) – consistency 위해
- ☐ OpenVINS 코드 구조: Propagator, UpdaterSLAM, UpdaterMSCKF, StateHelper
- ☐ Seg-aided plane measurement 을 UpdaterMSCKF 와 평행하게 끼우는 디자인

연구 차별점 (ov_plane 한계 → 본 연구 강점)

1. ov_plane: 흰 벽 / 동적 환경 약함 → 본 연구: stereo+IMU gravity 로 robust
2. ov_plane: $O(n^2)$ plane state → 본 연구: segmentation 으로 instantaneous plane (state 에 안 넣음)
3. ov_plane: indoor 중심 → 본 연구: KITTI outdoor 검증

5.4 다음 1 주 작업 흐름 (제안)

Day 1-2: ov_plane Propagator.cpp 읽기 (IMU predict + cloning 흐름)
 Day 3-4: ov_plane UpdaterMSCKF.cpp 읽기 (null-space, chi-square)
 Day 5: 본 연구 plane measurement 를 어디에 끼울지 디자인 메모
 Day 6-7: 작은 prototype (state cloning 만 본 repo 에 이식)

부록 A — PDF 추출 방법 3 가지

본 문서는 마크다운으로 작성됨. 시스템에 PDF 도구가 없어 다음 중 택1.

A-1. VS Code "Markdown PDF" extension (권장 — 5 분)

1. VS Code → Extensions → "Markdown PDF" (yzane) 검색·설치
2. 본 .md 파일 열기 → 우클릭 → "Markdown PDF: Export (pdf)"
3. 같은 폴더에 .pdf 생성됨

수식 렌더링: 본 extension 은 MathJax 기본 지원. $\dots$ 인라인, $\$ \$ \dots \$ \$$ 블록 모두 OK.

A-2. 브라우저 인쇄 (도구 0)

1. 본 .md 파일을 GitHub/GitLab 에 push (수식 자동 렌더)
또는 VS Code 의 "Markdown All in One" extension 으로 preview 열기
2. Preview 패널 또는 GitHub 페이지 → Ctrl+P → "Save as PDF"
3. 여백 / 배율 조정 후 저장

A-3. Pandoc (LaTeX 수준 출력, 설치 5~10 분)

1. Pandoc 설치 (winget 또는 <https://pandoc.org>)
 winget install --id JohnMacFarlane.Pandoc

2. MiKTeX 설치 (LaTeX 엔진, ~500MB)
 winget install --id MiKTeX.MiKTeX

3. 변환

```
pandoc docs/insight/20260520_vio_lvio_msckf_thinking_cheatsheet.md `
-o docs/insight/20260520_vio_lvio_msckf_thinking_cheatsheet.pdf `
--pdf-engine=xelatex `
-V geometry:margin=2cm `
-V mainfont="Malgun Gothic" `
```

--toc

한글 폰트는 Malgun Gothic (Windows 기본). LaTeX 첫 실행 시 패키지 다운로드 시간 걸림.

부록 B — 추가 학습 자료 (선택, 평생 reference)

자료	가치
Barfoot, State Estimation for Robotics	EKF / MSCKF / Lie group 완전체. 무료 PDF 공개
Sola, Quaternion kinematics for ESKF (arXiv:1711.02508)	Error-state EKF 의 정전 (定典). 본 연구 LC-EKF 5
Geneva et al., OpenVINS (ICRA 2020)	MSCKF + plane (ov_plane) 의 reference codeba
Mourikis & Roumeliotis, MSCKF original (ICRA 2007)	Null-space projection 의 출처. 10 페이지.
Forster et al., On-Manifold Preintegration	IMU preintegration 의 정전. VINS-Mono 의 출처
Cyrill Stachniss YouTube — Mobile Sensing and Robotics	EKF/UKF 강의 베스트. 무료.

부록 C — 관련 메모리 & 본 repo 산출물

- `[[user_learning_context]]` C++ + VIO/EKF 병행 학습
- `[[project_main_research]]` 본 졸업 연구 (Seg-aided TC MSCKF)
- `[[project_tc_msckf_pivot]]` LC → TC 전환 결정
- `[[reference_ov_plane]]` ov_plane 논문 한계 & 본 연구 차별점
- `[[feedback_doc_structure]]` 본 문서의 다층 블록 구조 출처

본 repo 내 reference:

- `docs/insight/20260514_lc_ekf_cam_imu_fusion.md` — 본인이 작성한 LC-EKF 직관 노트
 - `include/ekf/lc_ekf.hpp`, `src/ekf/lc_ekf.cpp` — 본 연구 EKF 구현
 - `include/ekf/imu_propagator.hpp` — IMU predict 구현
-

작성 의도: 이 문서는 지식의 압축 이 아니라 카탈로그의 외부 저장소. 새 알고리즘 만날 때마다 어느 슬롯에 속하나 를 본 문서에 추가하면, 1 년 뒤 본인 머릿속 카탈로그가 이 문서의 두께만큼 두꺼워져 있을 것.